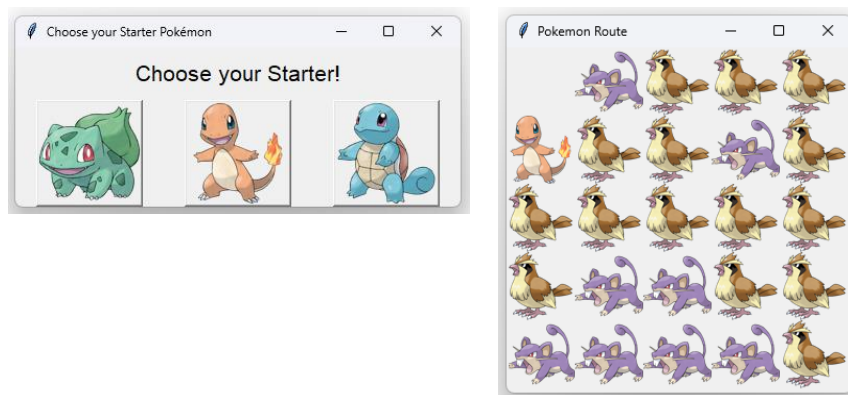


Pokémon Route

Implementar um jogo que utiliza Prolog como base de conhecimento e um sistema de regras baseado em lógica Fuzzy para tomar decisões.

Neste jogo existem routes, compostas por vários Pokémons, de vários níveis, definidas em factos Prolog. O utilizador escolhe o Pokémon inicial e este, utilizando um sistema de lógica Fuzzy, decide qual o próximo Pokémon que vai atacar. Este será o Pokémon, dos que se situam na sua vizinhança (exceto na diagonal), com a qual o Pokémon inicial tem uma maior probabilidade de ganhar o ataque. O jogo termina quando o Pokémon inicial perder uma batalha ou completar/terminar a route.



Na pasta “pokemon_route.zip”, existe a estrutura de ficheiros descrita a baixo. Onde está indicado **# TO DO** é onde será necessário completar.

```
pokemon_route/  
├── main.py           # to run the game  
├── game/  
│   ├── pokemon_game.py # game motor # TO DO  
│   └── pokemon_fuzzy.py # to define the Fuzzy logic # TO DO  
├── gui/  
│   ├── pokemon_gui.py  # GUI for the Pokémon route  
│   └── starter_gui.py  # GUI to choose the Pokémon starter  
├── prolog/  
│   ├── pokemon_route.pl # facts about the route  
│   ├── pokemon_game.pl  # rules about the route # TO DO  
│   ├── pokemon_info_attacks.pl # facts about the attacks' effects  
│   └── pokemon_list.pl   # facts about 150 Pokémon  
└── images/  
    ├── bulbasaur.png  
    ├── charmander.png  
    ├── squirtle.png  
    ├── pidgey.png  
    └── rattata.png
```

No main (main.py) é criada a GUI para escolher o Pokémon inicial (starter_gui.py), de seguida é criado um game (pokemon_game.py) utilizando o id do Pokémon inicial (starter_id) e a GUI do jogo que representa a route (pokemon_gui.py). O Pokémon inicial, começa o jogo com 4 de nível.

A GUI do jogo consulta o ficheiro Prolog, pokemon_game.pl, para obter informação sobre a route. A route está armazenada num facto **route(M)**, no ficheiro pokemon_route.pl, tal que M é uma matriz de pares, e cada par (Id,Level) corresponde ao id do Pokémon nessa posição e ao seu nível. Através dessa matriz, é desenhada a route na GUI. É também consultado o ficheiro Prolog pokemon_list.pl para obter o nome do Pokémon e a lista dos seus tipos. Tal informação está armazenada em factos **pokemon(Id,Name,Types)**, tal que Id é o identificador, Name o nome e Types uma lista com os tipos de ataque desse Pokémon.

Exemplo dos factos:

```
route([[ (0,0), (19,2), (16,2), (16,3), (16,6) ],
        [ (16,2), (16,3), (16,2), (19,3), (16,3) ],
        [ (16,3), (16,2), (16,3), (16,2), (16,5) ],
        [ (16,2), (19,2), (19,5), (16,3), (16,7) ],
        [ (19,2), (19,2), (19,2), (19,4), (16,7) ]]).

pokemon(16,  pidgey, [normal, flying]).
pokemon(19,  rattata, [normal]).
```

O par (0,0), representa a posição inicial do Pokémon, esta informação também está armazenada no facto **player_starts(X,Y)**.

Note que o ficheiro Prolog pokemon_game.pl possui ensure_loaded dos outros três ficheiros Prolog. Como tal, basta consultar esse para obter todos os factos presentes nos outros ficheiros.

Após estar a GUI carregada com a informação, o Pokémon inicial irá avaliar os Pokémons vizinhos (cima, baixo, esquerda e direita) e escolher qual o que vai atacar.

Caso o Pokémon vença, o seu nível poderá aumentar. Se o nível do Pokémon vencedor for inferior ao nível do Pokémon derrotado, o nível aumenta automaticamente em uma unidade. Se o nível do Pokémon vencedor for superior ao do adversário, existe uma probabilidade de 30% de o nível aumentar em uma unidade. Por fim, se ambos os Pokémon tiverem o mesmo nível, o aumento de uma unidade ocorre com uma probabilidade de 50%. Após derrotar um Pokémon, o Pokémon inicial continua a explorar até o jogo terminar, mas, para isso acontecer, é necessário implementar os seguintes passos:

- 1) No pokemon_game.py, a função evaluate_next_rooms(), utilizada em next_step(), consulta o ficheiro Prolog através do predicado **next_rooms(X,Y,Rooms)**, tal que X e Y são as coordenadas da posição do Pokémon inicial, e Rooms é uma lista de listas, em que cada lista contém a informação dos

Pokémons vizinhos [Id, Name, Level, X, Y, Types], ou seja, o identificador, o nome, o nível, as coordenadas da posição e ainda a lista de tipos desse Pokémon.

```
?-next_rooms(0, 0, Rooms).
Rooms = [ [19, rattata, 2, 1, 0, [normal]],
          [16, pidgey, 2, 0, 1, [normal, flying]] ]
```

É necessário implementar este predicado no ficheiro `pokemon_game.pl`, mantendo o formato de `Rooms` exemplificado **# TO DO**. Pode criar predicados auxiliares ao predicado **next_rooms**, mas não pode utilizar nenhum mecanismo do Prolog que não tenha sido lecionado em aula.

2) No `pokemon_game.py`, na função `evaluate_next_rooms()`, é chamada a função `calculate_prob(level_input, effect_input)`, do ficheiro `pokemon_fuzzy.py`. Esta devolve a probabilidade do Pokémon inicial ganhar a outro Pokémon e recebe:

- `level_input`, a diferença entre o nível do Pokémon inicial e o nível do Pokémon a atacar.
- `effect_input`, o efeito do ataque entre os dois Pokémon.

Este valor é calculado na função `total_attack(attacker_types, defender_types)`, no `pokemon_game.py`, que necessita de ser implementada **# TO DO**.

O cálculo do ataque é feito da seguinte forma:

- É necessário consultar o ficheiro Prolog para obter o efeito de cada tipo de ataque. Estes estão armazenados em factos, **attack(AttackerType, DefenderType, Effect)**, como por exemplo: `attack(normal, normal, 1)`, o que significa que se um Pokémon com o tipo `normal` atacar outro com o tipo `normal` tem um efeito com o valor 1.

Note que os efeitos dos ataques podem ser {0,0.5,1,2}.

- Será escolhido o tipo de ataque que tiver mais efeito combinado (multiplicando o valor dos efeitos) contra os tipos do outro Pokémon. Por exemplo:

```
attacker_types = [grass, poison]
# do Pokémon que vai atacar
defender_types = [normal, flying]
# do Pokémon que será atacado
```

```
attack( grass, normal, 1).
attack( grass, flying, 0.5).
attack( poison, normal, 1).
attack( poison, flying, 1).
```

Atacar com grass = $1 * 0.5 = 0.5$, e atacar com poison = $1 * 1 = 1$.
Como tal o ataque escolhido é do tipo poison, e o efeito do ataque é de 1.

Esta função, `calculate_prob(level_input, effect_input)`, necessita de ser implementada **# TO DO**. Para tal é necessário criar um sistema de regras Fuzzy no ficheiro `pokemon_fuzzy.py`. Este sistema utiliza como input das regras o `level_input` e o `effect_input`, explicados acima, e devolve como output a probabilidade de ganhar, em valores entre 0 e 1. A ideia geral é que quanto melhor é o Pokémon inicial em comparação com o outro, tanto em nível como em efeito do ataque, maior é a probabilidade de ganhar. Deverão ser definidos os universos de discurso, funções de pertença e regras Fuzzy adequadas ao problema.

Em resumo, para o jogo funcionar, é necessário implementar:

- No ficheiro `pokemon_game.pl`, o predicado **`next_rooms(X,Y,Rooms)`**
- No ficheiro `pokemon_fuzzy.py`, o sistema de regras fuzzy e o método `calculate_prob(level_input, effect_input)`
- No ficheiro `pokemon_game.py`, a função `total_attack(attacker_types, defender_types)`

Entrega:

O trabalho tem de ser realizado em grupos de 2. Tem de ser entregue um ficheiro zip com todos os ficheiros deste projeto, até às 23:59 do dia 29 de março no Moodle.

O ficheiro zip tem de possuir como nome `Nome1_Número1_Nome2_Número2.zip` (por exemplo: `MariaSilva_123456_JoaoLopes_654321.zip`). É obrigatório manter os nomes dos ficheiros, métodos e predicados pedidos para a implementação pois serão realizados testes automáticos.